

Dockerfile:

- > A Dockerfile is a text file that contains step-by-step instructions on how to build a Docker image.
- > Without a Dockerfile, you would have to manually:
 - > Install dependencies
 - > Configure the environment
 - > Copy code
 - > Run setup commands
- > A Dockerfile automates all this.

Basic Example:

```
FROM ubuntu:latest
WORKDIR /home/app
COPY . /home/app
RUN apt update && apt install openjdk-17-jdk -y
CMD ["echo", "hello"]
```

- > To build image using Dockerfile: `docker build -t sample:v1.0 .`
- > To run the container from the image: `docker run -it sample`

Common instructions in a Dockerfile:

1. **FROM:** Defines the base image from which your Docker image will be built.

Syntax: `FROM <image>:<tag>`
Example: `FROM ubuntu:22.04`
Meaning: Build your image on top of Ubuntu 22.04.

2. **WORKDIR:** Sets the working directory inside the image. All following commands run from this location.

Syntax: `WORKDIR /path`
Example: `WORKDIR /app`
Meaning: If /app does not exist, Docker will create it.

3. **COPY:** Copies files from your host system into the image.

Syntax: `COPY <source> <destination>`
Example: `COPY . /app`
Meaning: Copy everything from your current directory to /app inside the image.

4. **ADD:** Similar to COPY but supports URLs and automatically extracts tar archives.

Syntax: `ADD <source> <destination>`
Example (URL): `ADD https://example.com/file.tar.gz /opt/`
Example (local tar): `ADD app.tar.gz /app`
Note: Use COPY unless you specifically need ADD features.

5. **RUN:** Executes commands at image build time (not when the container runs).

Syntax: `RUN <command>`
Example: `RUN apt update && apt install -y openjdk-17-jdk`
Meaning: Installs Java inside the image during build.

6. **CMD:** Specifies the default command that runs when the container starts.

Syntax: `CMD ["executable", "arg1", "arg2"]`
Example: `CMD ["python3", "app.py"]`
Note:

- > Only one CMD instruction is allowed.
- > CMD is overridden if you pass a command in docker run.

7. **ENTRYPOINT:** Similar to CMD but cannot be easily overridden. It defines the "main executable" of the container.

Syntax: `ENTRYPOINT ["executable", "arg"]`
Example: `ENTRYPOINT ["java", "-jar", "app.jar"]`
Meaning: The container will always run this Java application.

8. **EXPOSE:** Documents which port the container listens on. It does not publish the port by itself.

Syntax: `EXPOSE <port>`
Example: `EXPOSE 8080`

- > To actually map port:
- > `docker run -p 8080:8080 image`

9. **ENV:** Sets environment variables inside the container.

Syntax: `ENV KEY=value`
Example: `ENV APP_ENV=production`
Meaning: Inside the container:
`echo $APP_ENV`
`production`

10. **VOLUME:** Used to provide a dedicated storage for storing the data of the containers. This data will stay even if the container gets deleted.

Syntax: `VOLUME ["/path"]`
Example: `VOLUME ["/var/lib/mysql"]`
Meaning: This directory will store data even after the container is deleted.

11. **USER:** It is used to change the user under which the commands run.

Syntax: `USER <username or UID>`
Example: `USER appuser`
Meaning: Commands now run as appuser instead of root.

12. **LABEL:** Adds metadata to your image.

Syntax: `LABEL key=value`
Example: `LABEL maintainer="sumukha"`

13. **ARG:** Defines variables available only during the build stage.

Syntax: `ARG <name>=<default>`
Example: `ARG VERSION=1.0`
Using ARG inside Dockerfile: `ENV APP_VERSION=$VERSION`

14. **HEALTHCHECK:** This command is used to check the health of the container.

Syntax: `HEALTHCHECK CMD <command>`
Example: `HEALTHCHECK --interval=30s --timeout=5s --retries=3 \`
`CMD curl -f http://localhost:8080/actuator/health || exit 1`
Meaning: If the command fails, Docker marks the container as unhealthy.

15. **SHELL:** It is used to specify the shell to be used inside the container.

Syntax: `SHELL ["shell", "option"]`
Example: `SHELL ["/bin/bash", "-c"]`
note:

- > /bin/bash → the shell you are using
- > -c tells bash to run the command you pass next.

To push the Docker images to DockerHub

- 1) Go to a browser and search for "Docker Hub" and login.
- 2) On the homepage you will be getting an option called as "Visit catalog now" click on it.
- 3) Then in the sidebar you will be getting an option called as "Repositories" click on it.
- 4) Here you will be getting a button called as "Create a Repository" click on it.
- 5) Give a repository name and description and then choose public or private based on your needs and then click on create. This is how you create your own repository.
- 6) After creating the repository. Type these 3 commands to push the images to the repository.
`docker login` This command is used to login to Docker Hub from the terminal.
`docker tag <image-name> <dockerhub_ID>/<image-name>:version` This command is used to give a tag to the image.
`docker push <dockerhub_ID>/<image-name>:version` This command is used to push the image to Docker Hub.

Example for static-web-hosting:

- 1) Create a folder and then open VScode from that folder.
- 2) To clone the static website from the GitHub, write these commands:

```
> git clone https://github.com/Sumukha47/Aqua-nova.git
> cd Aqua-nova
```
- 3) Create a file called as "Dockerfile" and then write these commands:
`FROM nginx:latest` → Base image of nginx
`COPY . /usr/share/nginx/html` → Copies the static website files into the nginx directory.
`EXPOSE 80` → Defining the default port number for nginx.
- 4) After writing these commands in the Dockerfile, save it and then write these docker commands to create an image, run container and then save the image in the repository (Run these commands in the VS code terminal):

```
> docker build -t static-web:latest .
> docker run -d -p 80:80 --name static-web static-web:latest
> docker login
> docker tag static-web:latest sumukha200147/sumukha-myimg:latest
> docker push sumukha200147/sumukha-myimg:latest
```

Example for spring-petclinic:

- 1) Create a folder and then open VScode from that folder.
- 2) To clone the spring-petclinic repository, write these commands:

```
> git clone https://github.com/spring-projects/spring-petclinic.git
> cd spring-petclinic
```
- 3) Create a file called as "Dockerfile" and then write these commands:
`FROM maven:3.8.5-eclipse-temurin-17` → Base image of maven
`WORKDIR /app` → Creates a separate directory called as app. all the other commands will be executed within this directory.
`COPY pom.xml /app` → Copies the pom.xml into the working directory.
`COPY src /app/src` → Copies the src into the working directory and within that inside src folder.
`RUN mvn clean package -DskipTests` → Creates a .jar file and skips test phase.
`EXPOSE 8080` → Container port 8080 (default port for spring-petclinic)
`CMD mv ./target/*.jar app.jar && java -jar app.jar` → Command to rename the .jar file as app.jar and execute jar file.
- 4) After writing these commands in the Dockerfile, save it and then write these docker commands to create an image, run container and then save the image in the repository:

```
> docker build -t petclinic:latest .
> docker run -d -p 80:8080 --name petclinic petclinic:latest
> docker login
> docker tag petclinic:latest sumukha200147/spring-petclinic:latest
> docker push sumukha200147/spring-petclinic:latest
```

Note: If we have multiple Dockerfiles (like: Dockerfile.dev, Dockerfile.test)
then to specifically run a Dockerfile, we use the command:

```
docker build -f Dockerfile.dev -t myapp-dev .
docker build -f Dockerfile.test -t myapp-test .
```